Text-Based Prediction of Book Review Popularity

Complete Project Architecture, Feature Engineering, Hyperparameter Rationale & Viva Defense Guide

Course: UE24CS352A Machine Learning | Problem Statement #41 | Reference: Stanford CS229 (Bridget Daly) | Team: Preksha C Mudigoudar & Team

1. The Core Problem & The 30-Second Elevator Pitch

• **The Platform & Phenomenon:** On Goodreads, book review engagement follows an extreme power-law distribution. Approximately 79% of reviews receive virtually zero attention, while the top ~21% capture the overwhelming majority of community discussion and likes.
• **The Machine Learning Objective:** Predict whether a newly posted book review will achieve substantial relative community engagement based on the review's text linguistics, reviewer reputation, and book contextual scale.
• **Mathematical Target Formulation:** Rather than using raw vanity like counts (which heavily penalize small indie books), engagement is measured relative to the book's total review activity:
  like_share = (review_likes + review_comments) / book_total_review_likes
  If **like_share > 0.02 (2%)**, the review is classified as **Popular (y = 1)**; otherwise **Unpopular (y = 0)**.
• **The 30-Second Viva Summary:** *"We replicated the Stanford CS229 benchmark on 100,000 Goodreads reviews, resolved the 79:21 class imbalance via Cost-Sensitive Loss Weighting (recovering 78.7% Sensitivity without discarding data), and engineered novel Contextual Scale interaction features that shattered the published 0.700 ROC-AUC ceiling to 0.842."*

2. Stanford Baseline (Bridget Daly, 2021) vs. Why It Hit a Wall at 0.700

• **Baseline Dataset:** UCSD Book Graph (Wan et al.) — filtered down to 1.98M reviews across 54k books.
• **Baseline Feature Tiers:**
  • *Subset A (Metadata — 4 features):* user_reviews, days_since_review, user_rating, rating_diff.
  • *Subset B (Metadata + NLP — 12 features):* Subset A + word count, sentence length, word length, Penn-Treebank POS fractions (verbs, nouns, adjectives), quotation flag, and VADER compound sentiment.
  • *Subsets C & D (Vocabulary — 10,012 features):* Subset B + top 10,000 Bag-of-Words (C) or TF-IDF tokens (D).
• **Why the Paper Hit a Performance Ceiling (ROC-AUC ≈ 0.700):**
  1. **The Contextual Scale Blind Spot:** The Stanford author analyzed text and reviewer counts in a vacuum, completely omitting **book-level audience scale** (*ratings_count*). Having 50 past reviews on an indie book makes you a dominant influencer; having 50 reviews on Harry Potter leaves you completely invisible.
  2. **Destructive Random Undersampling:** The paper discarded ~70% of majority class training observations to force a 50:50 ratio, throwing away valuable variance and real-world boundary information.
  3. **Extreme Skewness:** The paper fed raw counts into linear models without addressing exponential power-law distributions.

3. Our Novel Feature Engineering Suite: The 23 Contextual Features

To break the 0.700 ceiling, we engineered **Subset Context (23 features)**, transforming Pearson correlation with popularity from +0.22 to +0.47:

Feature Name	Category	Mathematical Formula / Definition	Predictive Signal & Real-World Intuition
user_to_book_ratio	Relative Scale	ln(1 + user_reviews) / [ln(1 + ratings_count) + 1e-5]	**Single strongest feature (|r| = 0.472).** Measures user prominence relative to book audience size. Ratio > 0.8 indicates dominant platform voice.
contrarian_score	Controversy Tension	|user_rating - avg_rating| × ln(1 + num_words)	Captures debate tension (|r| = 0.385). Long reviews sharply disagreeing with community consensus generate heavy comments and viral upvotes.
review_effort	Text Structure	ln(1 + num_words) × (1 + quote)	Rewards comprehensive critiques containing direct book quotations. High effort strongly correlates with community trust and saves.
rating_extremity	Psychological	1 if user_rating ∈ {1.0, 5.0} else 0	Extreme praise (5★) or harsh criticism (1★) polarizes readers and drives significantly higher engagement than moderate 3★ reviews.
log_ratings_count	Audience Scale	ln(1 + ratings_count)	Compresses book audience scale from 0–4,000,000 into a smooth 0–16 Gaussian curve, eliminating massive power-law outlier distortion.
log_user_reviews	Authority Scale	ln(1 + user_reviews)	Reflects diminishing returns: jumping from 1 to 20 reviews provides huge authority gains; jumping from 2,000 to 2,020 is negligible.
log_num_words	Linguistic Scale	ln(1 + num_words)	Normalizes review length. Distinguishes 10-word one-liners from 300-word essays without letting 2,000-word outliers skew model weights.

log_days	Temporal	ln(1 + days_since_review)	Models visibility accumulation: 10 days early at launch week matters enormously; 10 days difference after 3 years is completely irrelevant.
abs_rating_diff	Sentiment Gap	|user_rating - avg_rating|	Magnitude of opinion divergence from the community average regardless of positive/negative direction.
sentiment (VADER)	NLP Sentiment	VADER Compound polarity score [-1.0, +1.0]	Captures emotional intensity. Passionate reviews (highly positive or critical) outperform neutral, lukewarm reviews.
pct_verbs, nouns, adj	Penn-Treebank POS	NLTK POS Tag ratios / total words	Measures syntactic composition and descriptive richness. High adjective/noun density signals informative critique.

★ Why do we take the natural logarithm ln(1 + x) everywhere? (Crucial Viva Question)

1. **Resolves Extreme Skewness:** Web engagement metrics follow exponential power laws. Raw ratings_count ranges from 5 to 4,000,000+ (a 800,000x gap!). Without log transforms, 99.2% of data is crushed into the first histogram bin, causing gradient explosions in neural networks and extreme leverage in linear models.
2. **Models Real Human Diminishing Returns:** A 10-day difference during book launch week is massive; a 10-day difference after 3 years is zero. Log transforms mathematically replicate this human cognitive reality.
3. **Enables Stable Ratios:** Taking ratios of logs creates smooth, normalized indicators (like user_to_book_ratio from 0.0 to 1.2).

4. Implemented Models & Rigorous Hyperparameter Choices

Every single model and hyperparameter in our codebase was selected based on computational constraints, mathematical theory, and syllabus alignment (Units 1, 2, and 3):

Model Architecture	Algorithm & Library	Exact Hyperparameters Used	Engineering Justification & Syllabus Connection
Logistic Regression (Baselines & Balanced)	sklearn.linear_model LogisticRegression	• solver='saga' • penalty='l2' • C=1.0 • max_iter=1000 • class_weight='balanced'	• **Why SAGA?** Stochastic Average Gradient with Accelerated extensions. Unlike standard SGD which is noisy, or Batch GD which is slow on 100k rows, SAGA stores past gradients to achieve linear convergence on large, sparse matrices. • **Why L2 Penalty?** Penalizes sum of squared weights, preventing individual rare words from receiving massive overfitted coefficients. • **Why Balanced Weight?** Penalizes minority errors by N_neg / N_pos ≈ 3.76, shifting the linear decision threshold to recover sensitivity.
HistGradientBoosting (Contextual Breakthrough Champion)	sklearn.ensemble HistGradientBoosting Classifier	• max_iter=150–200 • max_depth=5–6 • learning_rate=0.08 • sample_weight=3.76 • random_state=229	• **Why Hist (Histogram-based)?** Bins continuous features into 256 discrete integer bins. Reduces split finding from O(N log N) to O(N), running 20x–50x faster with zero loss in mathematical precision. • **Why max_depth=5–6?** Student Experiment 2 proved single trees peak in generalization at d=7. Constraining boosting base trees to depth 5–6 ensures shallow weak learners that combine smoothly without overfitting. • **Why learning_rate=0.08?** Shrinkage factor. A conservative learning rate prevents overshooting loss minima across 150+ sequential trees.
Stacking Ensemble Meta-Learner (Production High-Specificity)	sklearn.ensemble StackingClassifier (Unit 3 Syllabus)	• Level 1 Base Learners: - LR (class_weight='balanced') - HistGB (depth=5, iter=120) - Random Forest (100 trees, depth=8) • Level 2 Meta-Classifier: - LogisticRegression() • cv=3, stack_method='predict_proba'	• **Why Stacking?** Fulfills Unit 3 (Ensemble Learning). Combines three fundamentally diverse hypothesis spaces: linear hyperplanes (LR), gradient-boosted residuals (GB), and bootstrap aggregation (Random Forest). • **Why LR as Meta-Learner?** Learns optimal calibrated weights for base model probabilities without introducing high non-linear variance. • **Why cv=3?** Uses 3-fold cross-validation out-of-fold predictions to prevent severe data leakage between base estimators and meta-learner. • **Result:** Attains highest overall accuracy (82.5%) and 93.7% Specificity.
PyTorch ReviewMLP (Deep Feedforward Neural Net)	torch.nn & torch.optim Custom ReviewMLP (Unit 2 Syllabus)	• Input(23) → Linear(32) → ReLU → Dropout(0.2) → Linear(16) → ReLU → Linear(1) • Loss: BCEWithLogitsLoss(pos_weight=3.76) • Optimizer: Adam(lr=0.001) • batch_size=256, epochs=30	• **Why 32→16 Architecture?** Student Experiment 1 proved a 2-layer MLP matches or exceeds a 3-layer net (0.836 Val AUC) while training in half the time with far fewer parameters. • **Why Dropout(0.2)?** Randomly deactivates 20% of hidden neurons during each forward pass, breaking feature co-adaptation and preventing overfitting. • **Why BCEWithLogitsLoss?** Combines Sigmoid activation and Binary Cross-Entropy into a single numerically stable formula using the log-sum-exp trick, preventing float underflow/overflow (ln(0) crashes) while cost-weighting positive samples. • **Why Adam (lr=0.001)?** Adapts individual learning rates using first and second gradient moments, ensuring stable convergence across heterogeneous tabular scales.

★ How is Random Forest Built Inside the Stacking Classifier? (Unit 3 Core Topic)

1. **Row Bagging (Bootstrap Aggregation):** For 100 trees, 100 distinct training sets of size N are sampled *with replacement* from the 70,000 training reviews. Each tree sees a slightly different permutation of samples.
2. **Column Bagging (Random Feature Subsets):** At every single split node, the tree is restricted to evaluating only a random subset of features ($\sqrt{23} \approx 5$ features). This prevents dominant features (like user_to_book_ratio) from appearing at the root of every tree, forcing different trees to learn diverse patterns.
3. **Ensemble Averaging:** Individual tree predictions are averaged, reducing model variance by $1/\sqrt{M}$ while maintaining low bias.

5. Empirical Benchmark Results & The Accuracy Paradox

All models evaluated on the untouched 15% holdout test partition (15,000 reviews: 11,850 unpopular, 3,150 popular). Ground truth results saved in *goodreads_ML_plus/master_results_table.csv*:

Feature Subset	Model Architecture	Imbalance Strategy	Accuracy	Sensitivity	Specificity	ROC-AUC
Subset A (Meta)	Logistic Regression	Baseline (Unweighted)	79.0%	6.5%	98.3%	0.670
Subset B (Meta+NLP)	Logistic Regression	Baseline (Unweighted)	79.4%	9.5%	98.0%	0.703
Subset B (Meta+NLP)	Logistic Regression	Random Undersample	69.2%	55.7%	72.7%	0.705
Subset B (Meta+NLP)	**Logistic Regression**	**Balanced Loss Weight**	69.2%	55.6%	72.9%	**0.706**
Subset B (Meta+NLP)	Gradient Boosting	Baseline (Unweighted)	80.4%	16.1%	97.5%	0.730
Subset B (Meta+NLP)	Gradient Boosting	Random Undersample	66.9%	65.9%	67.1%	0.729
Subset B (Meta+NLP)	**Gradient Boosting**	**Cost-Sensitive (w=3.76)**	67.4%	64.9%	68.1%	**0.730**
Subset D (TF-IDF)	Gradient Boosting	TF-IDF 2000 (Cost-Sens)	68.1%	65.1%	68.9%	0.736
Subset Context (23f)	**Gradient Boosting (Context)**	**Cost-Sensitive (w=3.76)**	**74.6%**	**78.7%**	**73.6%**	**0.842**
Subset Context (23f)	**Stacking Meta-Learner**	**Ensemble (LR+GB+RF)**	**82.5%**	40.3%	**93.7%**	**0.841**
Subset Context (23f)	**PyTorch ReviewMLP**	**Cost-Weighted (w=3.76)**	**74.2%**	**78.8%**	73.0%	**0.841**

Core Takeaways from Experimental Measurements:

1. **The Accuracy Paradox in Action:** Baseline unweighted models hit ~80% accuracy, but Sensitivity is catastrophic (6.5% to 16.1%)! The model simply guesses 'Unpopular' almost every time. In an information retrieval application, missing 90% of viral reviews renders the system useless.
2. **Cost-Sensitive Equivalence Without Data Loss:** Cost-sensitive loss weighting matches or exceeds random undersampling performance (0.730 vs 0.729 AUC; higher specificity 68.1% vs 67.1%) while keeping **100% of the training dataset** and avoiding the deletion of ~36,000 majority reviews.
3. **The Contextual Breakthrough:** Shifting from Subset B (12 features) to Subset Context (23 features) drives ROC-AUC from **0.730 straight to 0.842**, while Sensitivity surges to **78.7%** (catching ~8 out of 10 viral reviews).
4. **Stacking Ensemble Superiority (Unit 3):** Blending LR, HistGB, and Random Forest delivers the highest overall accuracy (**82.5%**) and exceptional specificity (**93.7%**), filtering out false alarms.

6. Curricular Student Experiments (Syllabus Units 1 & 2)

• **Experiment 1 (Unit 2 — ANN Architecture Depth & Learning Rate Sweep):**
Tested 1-Layer (16 units), 2-Layer (32→16 + Dropout 0.2), and 3-Layer (64→32→16) on Contextual features under $\eta \in \{0.01, 0.001\}$.
Finding: 1-Layer was under-parameterized (Val AUC: 0.812). Both 2-Layer and 3-Layer networks converged stably to **0.835 – 0.836 Val AUC**. The 2-Layer ReviewMLP with 20% Dropout was proven optimal: it delivers maximum generalization with half the parameters and training time of the 3-Layer net.

• **Experiment 2 (Unit 1 — Empirical Decision Tree Bias-Variance Tradeoff Curve):**
Swept Decision Tree depth from d = 1 to 15 on Contextual features.
Finding: Demonstrates textbook machine learning theory. At d=1 (stump), the tree suffers from **high bias / underfitting** (Train AUC 0.710, Test AUC 0.710). As capacity increases, it reaches the **optimal generalization sweet spot at d = 7 (Test AUC: 0.824)**. Beyond $d \geq 12$, the model experiences **severe variance / overfitting**: Train AUC surges to 0.948 while Test AUC collapses to 0.694.

7. Anticipated Viva Q&A; — Top 8 Questions & Exact Answers

Q1: What is 'like_share' and why not use raw like counts?

Suggested Viva Answer: "Raw like counts are unfair across books. 50 likes on Harry Potter (which has 100,000 likes) is only 0.05% of engagement (Unpopular). But 50 likes on a niche indie book with 100 total likes is 50% of the book's total discussion (Highly Viral). like_share normalizes engagement by dividing review likes by the total likes across that entire book."

Q2: Why did the Stanford paper hit an AUC ceiling of 0.700, and how did you reach 0.842?

Suggested Viva Answer: "The Stanford paper analyzed review text in a vacuum without book-level scale. We engineered 'user_to_book_ratio' [ln(user_reviews)/ln(ratings_count)] and 'contrarian_score' [rating_diff × log_words]. This boosted feature correlation with popularity from +0.22 to

+0.47, driving ROC-AUC to 0.842."

Q3: Why use Cost-Sensitive Weighting instead of Random Undersampling?

Suggested Viva Answer: "Random undersampling discards ~70% of majority class training reviews to create a 50:50 ratio, destroying valuable variance and edge cases. Cost-sensitive weighting keeps 100% of data and sets a penalty weight w = N_neg / N_pos ≈ 3.76 on positive errors. It achieves identical or better AUC without throwing away 36,000 reviews."

Q4: Why did you take log(1 + x) transforms on ratings_count and user_reviews?

Suggested Viva Answer: "Real-world social engagement follows power laws. Book ratings range from 5 to 4,000,000+ (an 800,000x gap). Unscaled data crushes 99.2% of samples into the first bin and causes gradient explosions. Log transforms convert exponential data into smooth bell curves and model diminishing returns."

Q5: Why did you choose maximum tree depth of 5 to 7 in Gradient Boosting?

Suggested Viva Answer: "In Experiment 2, we swept tree depth from 1 to 15. The empirical bias-variance curve peaked at depth 7 (Test AUC: 0.824) and overfit heavily at depth 12+ (Train AUC 0.948 vs Test AUC 0.694). Setting max_depth=5–6 in boosting ensures base estimators remain shallow weak learners that combine smoothly without overfitting."

Q6: How is your Stacking Classifier built, and why does it achieve 82.5% accuracy?

Suggested Viva Answer: "Fulfilling Unit 3, it combines three distinct paradigms: Balanced Logistic Regression (linear boundary), HistGradientBoosting (non-linear thresholds), and Random Forest (bagging variance reduction). A Logistic Regression meta-classifier learns the optimal probability blending using 3-fold cross-validation out-of-fold predictions."

Q7: What does Reviewer Past Reviews = 30 mean in the interactive demo?

Suggested Viva Answer: "It represents the user's lifetime total reviews written across their entire Goodreads profile (not reviews on that single book, since Goodreads only permits 1 review per user per book). It indicates whether the reviewer is a casual reader (10–30 reviews) or a platform influencer (1,000+ reviews)."

Q8: How does the interactive demo run in under 0.1 seconds?

Suggested Viva Answer: "We serialized the trained Gradient Boosting model, StandardScaler, and the 23-feature schema into 'demo_model.pkl'. The script detects this binary artifact and loads it instantly without retraining, extracting NLTK POS tags and VADER scores on the fly from user text in real time."

Repository: https://github.com/Preksha-a11y/goodreads_ML | Master Runner: run_experiments.py | Interactive Demo: interactive_demo.py